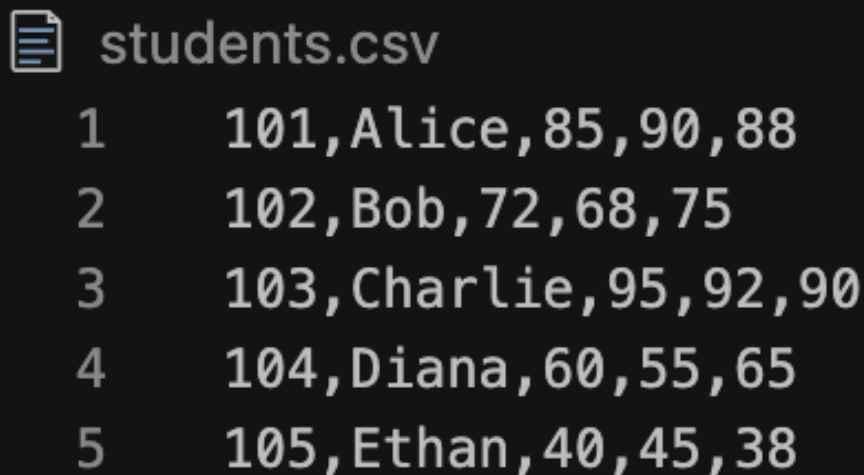


Name: Shreya Miriyam Varghese
Roll No: 2025BCS0034

IT WORKSHOP III LAB 18/08/2026

Q1.

Read student records containing roll number, name, and marks in multiple subjects. Calculate total, average, grade, and pass/fail status, then rank students and generate a separate result file.



```
students.csv
1    101,Alice,85,90,88
2    102,Bob,72,68,75
3    103,Charlie,95,92,90
4    104,Diana,60,55,65
5    105,Ethan,40,45,38
```

```
def calculate_result(student):
    marks = student["marks"]
    total = sum(marks)
    average = total / len(marks)
```

```

    if any(mark < 40 for mark in marks):
        status = "FAIL"
        grade = "F"
    else:
        status = "PASS"
        if average >= 90:
            grade = "A+"
        elif average >= 80:
            grade = "A"
        elif average >= 70:
            grade = "B"
        elif average >= 60:
            grade = "C"
        elif average >= 50:
            grade = "D"
        else:
            grade = "F"
    student["total"] = total
    student["average"] = average
    student["grade"] = grade
    student["status"] = status

def read_students():
    students = []
    file = open("students.csv", "r")
    for line in file:
        line = line.strip()
        data = line.split(",")
        student = {
            "roll": data[0],
            "name": data[1],
            "marks": [
                int(data[2]),
                int(data[3]),
                int(data[4])
            ]
        }
        calculate_result(student)
        students.append(student)
    file.close()
    return students

def assign_ranks(students):
    students.sort(

```

```

        key=lambda student: student["total"],
        reverse=True
    )

    rank = 1
    for student in students:
        student["rank"] = rank
        rank += 1

def display_results(students):
    print("\nSTUDENT RESULT")
    print("-" * 75)
    print(
        f"{'Rank':<6}"
        f"{'Roll':<8}"
        f"{'Name':<15}"
        f"{'Total':<8}"
        f"{'Average':<10}"
        f"{'Grade':<8}"
        f"{'Status':<8}"
    )
    print("-" * 75)
    for student in students:
        print(
            f"{student['rank']:<6}"
            f"{student['roll']:<8}"
            f"{student['name']:<15}"
            f"{student['total']:<8}"
            f"{student['average']:<10.2f}"
            f"{student['grade']:<8}"
            f"{student['status']:<8}"
        )

def write_results(students):
    file = open("student_result.csv", "w")
    file.write("STUDENT RESULT\n")
    file.write("-" * 75 + "\n")
    file.write(
        f"{'Rank':<6}"
        f"{'Roll':<8}"
        f"{'Name':<15}"
        f"{'Total':<8}"
        f"{'Average':<10}"
        f"{'Grade':<8}"

```

```

        f"{'Status':<8}\n"
    )

    file.write("-" * 75 + "\n")
    for student in students:
        file.write(
            f"{student['rank']:<6}"
            f"{student['roll']:<8}"
            f"{student['name']:<15}"
            f"{student['total']:<8}"
            f"{student['average']:<10.2f}"
            f"{student['grade']:<8}"
            f"{student['status']:<8}\n"
        )
    file.close()

students = read_students()
assign_ranks(students)

display_results(students)
write_results(students)
print("\nResult successfully saved in student_result.csv")

```

student_result.csv

1	STUDENT RESULT						
2	-----						
3	Rank	Roll	Name	Total	Average	Grade	Status
4	-----						
5	1	103	Charlie	277	92.33	A+	PASS
6	2	101	Alice	263	87.67	A	PASS
7	3	102	Bob	215	71.67	B	PASS
8	4	104	Diana	180	60.00	C	PASS
9	5	105	Ethan	123	41.00	F	FAIL

Q2.

Given a server log file, identify the number of INFO, WARNING, and ERROR messages, find the most frequent

IP address, count errors by type, and display lines containing critical errors.

```
server_log.txt
1 192.168.1.10 INFO Server started
2 192.168.1.20 INFO User logged in
3 192.168.1.10 WARNING High memory usage
4 192.168.1.30 ERROR File not found
5 192.168.1.20 INFO User logged out
6 192.168.1.30 ERROR File not found
7 192.168.1.40 ERROR Database connection failed
8 192.168.1.10 ERROR Database connection failed
9 192.168.1.30 ERROR CRITICAL Disk failure
10 192.168.1.20 WARNING Invalid login attempt
```

```
info = 0
warning = 0
error = 0

ip_count = {}
error_types = {}
critical_lines = []

file = open("server_log.txt", "r")

for line in file:

    line = line.strip()
    words = line.split()

    ip = words[0]
    level = words[1]

    if ip in ip_count:
        ip_count[ip] = ip_count[ip] + 1
    else:
        ip_count[ip] = 1

    if level == "INFO":
        info = info + 1
```

```

        elif level == "WARNING":
            warning = warning + 1

        elif level == "ERROR":
            error = error + 1

            error_type = " ".join(words[2:])

            if error_type in error_types:
                error_types[error_type] = error_types[error_type] +
1
            else:
                error_types[error_type] = 1

            if "CRITICAL" in line:
                critical_lines.append(line)

file.close()

most_ip = max(ip_count, key=ip_count.get)

print("LOG FILE ANALYZER")
print("-----")

print("INFO messages      :", info)
print("WARNING messages  :", warning)
print("ERROR messages     :", error)

print("\nMost Frequent IP Address:")
print(most_ip, "-", ip_count[most_ip], "times")

print("\nErrors by Type:")

for error_type in error_types:
    print(error_type, "-", error_types[error_type])

print("\nCritical Error Lines:")

for line in critical_lines:
    print(line)

result = open("analysis.txt", "w")

result.write("LOG FILE ANALYSIS\n")
result.write("-----\n")
result.write("INFO messages      : " + str(info) + "\n")

```

```

result.write("WARNING messages : " + str(warning) + "\n")
result.write("ERROR messages : " + str(error) + "\n")

result.write("\nMost Frequent IP Address:\n")
result.write(most_ip + " - " + str(ip_count[most_ip]) + "
times\n")

result.write("\nErrors by Type:\n")

for error_type in error_types:
    result.write(
        error_type + " - " +
        str(error_types[error_type]) + "\n"
    )

result.write("\nCritical Error Lines:\n")

for line in critical_lines:
    result.write(line + "\n")

result.close()

print("\nResult saved in analysis.txt")

```

```

analysis.txt
1  LOG FILE ANALYSIS
2  -----
3  INFO messages : 3
4  WARNING messages : 2
5  ERROR messages : 5
6
7  Most Frequent IP Address:
8  192.168.1.10 - 3 times
9
10 Errors by Type:
11 File not found - 2
12 Database connection failed - 2
13 CRITICAL Disk failure - 1
14
15 Critical Error Lines:
16 192.168.1.30 ERROR CRITICAL Disk failure

```

Q3.

Process a transaction file containing account number, transaction type, amount, and date. Calculate total deposits, withdrawals, current balance, largest transaction, and flag suspiciously large transactions.



transactions.csv

```
1  account,type,amount,date
2  1001,Deposit,50000,2026-08-01
3  1002,Deposit,25000,2026-08-02
4  1001,Withdrawal,10000,2026-08-03
5  1003,Deposit,75000,2026-08-04
6  1002,Withdrawal,5000,2026-08-05
7  1001,Deposit,150000,2026-08-06
8  1003,Withdrawal,20000,2026-08-07
9  1002,Deposit,30000,2026-08-08
```

```
def read_transactions():
    transactions = []

    file = open("transactions.csv", "r")

    # Skip header
    file.readline()
```



```
for line in file:
    line = line.strip()

    data = line.split(",")

    transaction = {
        "account": data[0],
        "type": data[1],
        "amount": float(data[2]),
        "date": data[3]
    }

    transactions.append(transaction)

file.close()

return transactions

def calculate_totals(transactions):
    total_deposits = 0
    total_withdrawals = 0

    for transaction in transactions:

        if transaction["type"] == "Deposit":
            total_deposits += transaction["amount"]

        elif transaction["type"] == "Withdrawal":
            total_withdrawals += transaction["amount"]

    balance = total_deposits - total_withdrawals

    return total_deposits, total_withdrawals, balance

def find_largest_transaction(transactions):
    largest = transactions[0]

    for transaction in transactions:

        if transaction["amount"] > largest["amount"]:
            largest = transaction

    return largest
```

```

def find_suspicious_transactions(transactions):
    suspicious = []

    for transaction in transactions:
        if transaction["amount"] > 100000:
            suspicious.append(transaction)

    return suspicious

def display_results(
    total_deposits,
    total_withdrawals,
    balance,
    largest,
    suspicious
):
    print("\nBANK TRANSACTION ANALYZER")
    print("-" * 50)

    print("Total Deposits      :", total_deposits)
    print("Total Withdrawals   :", total_withdrawals)
    print("Current Balance     :", balance)

    print("\nLargest Transaction:")
    print("Account  :", largest["account"])
    print("Type     :", largest["type"])
    print("Amount   :", largest["amount"])
    print("Date     :", largest["date"])

    print("\nSuspicious Transactions:")

    if len(suspicious) == 0:
        print("No suspicious transactions found.")

    else:
        for transaction in suspicious:
            print(
                transaction["account"],
                transaction["type"],
                transaction["amount"],
                transaction["date"]
            )

```

```

def main():
    transactions = read_transactions()

    total_deposits, total_withdrawals, balance = \
        calculate_totals(transactions)

    largest = find_largest_transaction(transactions)

    suspicious = find_suspicious_transactions(transactions)

    display_results(
        total_deposits,
        total_withdrawals,
        balance,
        largest,
        suspicious
    )

main()

```

BANK TRANSACTION ANALYZER

Total Deposits : 330000.0
 Total Withdrawals : 35000.0
 Current Balance : 295000.0

Largest Transaction:
 Account : 1001
 Type : Deposit
 Amount : 150000.0
 Date : 2026-08-06

Suspicious Transactions:
 1001 Deposit 150000.0 2026-08-06

Q4.

Process a CSV containing customer records. Detect missing fields, invalid email/phone formats, duplicate records, and malformed rows. Save valid records to cleaned.csv and invalid records to errors.csv.

 customers.csv

```
1 id,name,email,phone
2 101,Alice,alice@gmail.com,9876543210
3 102,Bob,bob@gmail.com,9876543211
4 103,Charlie,,9876543212
5 104,Diana,diana@gmail.com,98765
6 105,Ethan,ethan@gmail.com,9876543214
7 101,Alice,alice@gmail.com,9876543210
8 106,Frank,frank@gmail.com
9 107,George,george@gmail,9876543216
10 108,Helen,helen@gmail.com,9876543218
11
```

```
import csv

def valid_email(email):
    if "@" in email and "." in email:
        return True
    else:
        return False

def valid_phone(phone):
    if len(phone) == 10 and phone.isdigit():
        return True
    else:
```

```

        return False

def clean_data():

    valid_records = []
    invalid_records = []
    duplicate_records = []

    seen_ids = set()

    file = open("customers.csv", "r")

    reader = csv.reader(file)

    header = next(reader)

    for row in reader:

        try:
            if len(row) != 4:
                invalid_records.append(row)
                continue

            customer_id = row[0].strip()
            name = row[1].strip()
            email = row[2].strip()
            phone = row[3].strip()

            if (
                customer_id == ""
                or name == ""
                or email == ""
                or phone == ""
            ):
                invalid_records.append(row)
                continue

            if customer_id in seen_ids:
                duplicate_records.append(row)
                continue

            if not valid_email(email):
                invalid_records.append(row)
                continue

```

```
        if not valid_phone(phone):
            invalid_records.append(row)
            continue

        valid_records.append(row)

        seen_ids.add(customer_id)

    except Exception:
        invalid_records.append(row)

file.close()

file = open("cleaned.csv", "w", newline="")

writer = csv.writer(file)

writer.writerow(header)

for row in valid_records:
    writer.writerow(row)

file.close()

file = open("errors.csv", "w", newline="")

writer = csv.writer(file)

writer.writerow(header)

for row in invalid_records:
    writer.writerow(row)

for row in duplicate_records:
    writer.writerow(row)

file.close()

print("Data cleaning completed.")
print("Valid records   :", len(valid_records))
print("Invalid records :", len(invalid_records))
print("Duplicate records:", len(duplicate_records))
print("Saved valid data to cleaned.csv")
print("Saved errors to errors.csv")
```

```
clean_data()
```

 cleaned.csv

```
1 id,name,email,phone
2 101,Alice,alice@gmail.com,9876543210
3 102,Bob,bob@gmail.com,9876543211
4 105,Ethan,ethan@gmail.com,9876543214
5 108,Helen,helen@gmail.com,9876543218
```

 errors.csv

```
1 id,name,email,phone
2 103,Charlie,,9876543212
3 104,Diana,diana@gmail.com,98765
4 106,Frank,frank@gmail.com
5 107,George,george@gmail,9876543216
6
7
8 101,Alice,alice@gmail.com,9876543210
```

Q5.

Given a folder containing several text files, ask the user for a keyword and search for it across all files. Display the filename, line number, and matching line for every occurrence.

 file1.txt

```
1 Python is easy to learn.
2 File handling is important in Python.
3 Functions make programs easier.
```


 file2.txt

```
1 Python can work with text files.  
2 Python supports many useful functions.  
3 Searching files is a common task.
```

 file3.txt

```
1 Java is another programming language.  
2 Python is popular for data science.  
3 Text files can store information.
```

```
import os  
  
def search_keyword():  
    keyword = input("Enter keyword to search: ")  
    found = False  
    files = os.listdir()  
    for filename in files:  
        if filename.endswith(".txt"):  
            file = open(filename, "r")  
            line_number = 0  
            for line in file:  
                line_number = line_number + 1  
                if keyword.lower() in line.lower():
```

```
        print("\nFile:", filename)
        print("Line Number:", line_number)
        print("Line:", line.strip())

        found = True

        file.close()

    if found == False:
        print("\nKeyword not found in any file.")

search_keyword()
```

Enter keyword to search: Python

File: file2.txt

Line Number: 1

Line: Python can work with text files.

File: file2.txt

Line Number: 2

Line: Python supports many useful functions.

File: file1.txt

Line Number: 1

Line: Python is easy to learn.

File: file1.txt

Line Number: 2

Line: File handling is important in Python.

File: input.txt

Line Number: 1

Line: Python is easy. Python is powerful.

File: input.txt

Line Number: 2

Line: Python is widely used.

Q6.

Maintain book issue/return records in a file. Allow the user to search books, issue a book, return a book, calculate overdue fines, and update the file.

 books.txt

```
1 101,Python Programming,Available,None,0
2 102,Data Structures,Available,None,0
3 103,Database Management,Available,None,0
4 104,Computer Networks,Available,None,0
5 105,Operating Systems,Available,None,0
```

```
def display_books():
    file = open("books.txt", "r")

    print("\nBOOK LIST")
    print("-" * 70)

    for line in file:
        line = line.strip()

        if line == "":
            continue

        data = line.split(",")

        if len(data) != 5:
            continue

        print(
            "ID:", data[0],
            "| Book:", data[1],
            "| Status:", data[2]
        )
```

```

        file.close()

def search_book():
    keyword = input("Enter book name or book ID: ")

    file = open("books.txt", "r")

    found = False

    for line in file:
        line = line.strip()

        if line == "":
            continue

        data = line.split(",")

        if len(data) != 5:
            continue

        book_id = data[0]
        book_name = data[1]

        if keyword.lower() in book_name.lower() or keyword ==
book_id:

            print("\nBook Found")
            print("Book ID      :", data[0])
            print("Name          :", data[1])
            print("Status         :", data[2])
            print("Issued To      :", data[3])
            print("Overdue        :", data[4], "days")

            found = True

    file.close()

    if found == False:
        print("Book not found.")

def issue_book():
    book_id = input("Enter book ID to issue: ")
    student = input("Enter student name: ")

```

```
file = open("books.txt", "r")

lines = file.readlines()

file.close()

found = False
updated_lines = []

for line in lines:

    line = line.strip()

    if line == "":
        continue

    data = line.split(",")

    if len(data) != 5:
        continue

    if data[0] == book_id:

        found = True

        if data[2] == "Available":

            data[2] = "Issued"
            data[3] = student
            data[4] = "0"

            print("Book issued successfully.")

        else:
            print("Book is already issued.")

    updated_lines.append(",".join(data) + "\n")

file = open("books.txt", "w")

file.writelines(updated_lines)

file.close()

if found == False:
```

```
        print("Book ID not found.")

def return_book():
    book_id = input("Enter book ID to return: ")

    file = open("books.txt", "r")

    lines = file.readlines()

    file.close()

    found = False
    updated_lines = []

    for line in lines:

        line = line.strip()

        if line == "":
            continue

        data = line.split(",")

        if len(data) != 5:
            continue

        if data[0] == book_id:

            found = True

            if data[2] == "Issued":

                days = int(input("Enter number of overdue days:
"))

                fine = days * 5

                print("Overdue Fine: Rs.", fine)

                data[2] = "Available"
                data[3] = "None"
                data[4] = "0"

                print("Book returned successfully.")
```

```

        else:
            print("Book is already available.")

        updated_lines.append(",".join(data) + "\n")

    file = open("books.txt", "w")

    file.writelines(updated_lines)

    file.close()

    if found == False:
        print("Book ID not found.")

def main():

    while True:

        print("\n===== LIBRARY MANAGEMENT =====")
        print("1. Display Books")
        print("2. Search Book")
        print("3. Issue Book")
        print("4. Return Book")
        print("5. Exit")

        choice = input("Enter your choice: ")

        if choice == "1":
            display_books()

        elif choice == "2":
            search_book()

        elif choice == "3":
            issue_book()

        elif choice == "4":
            return_book()

        elif choice == "5":
            print("Program ended.")
            break

        else:
            print("Invalid choice.")

```

```
main()
```

```
===== LIBRARY MANAGEMENT =====
1. Display Books
2. Search Book
3. Issue Book
4. Return Book
5. Exit
Enter your choice: 2
Enter book name or book ID: Python

Book Found
Book ID   : 101
Name      : Python Programming
Status    : Available
Issued To : None
Overdue   : 0 days

===== LIBRARY MANAGEMENT =====
1. Display Books
2. Search Book
3. Issue Book
4. Return Book
5. Exit
Enter your choice: 1

BOOK LIST
-----
ID: 101 | Book: Python Programming | Status: Available
ID: 102 | Book: Data Structures | Status: Available
ID: 103 | Book: Database Management | Status: Available
ID: 104 | Book: Computer Networks | Status: Available
ID: 105 | Book: Operating Systems | Status: Available

===== LIBRARY MANAGEMENT =====
1. Display Books
2. Search Book
3. Issue Book
4. Return Book
5. Exit
Enter your choice: 3
Enter book ID to issue: 101
Enter student name: Shreya
Book issued successfully.

===== LIBRARY MANAGEMENT =====
1. Display Books
2. Search Book
3. Issue Book
4. Return Book
5. Exit
Enter your choice: 4
Enter book ID to return: 101
Enter number of overdue days: 0
Overdue Fine: Rs. 0
Book returned successfully.

===== LIBRARY MANAGEMENT =====
1. Display Books
2. Search Book
3. Issue Book
4. Return Book
5. Exit
Enter your choice: Exit
Invalid choice.

===== LIBRARY MANAGEMENT =====
1. Display Books
2. Search Book
3. Issue Book
4. Return Book
5. Exit
Enter your choice: 5
Program ended.
```